

Patient Health Monitoring Pipeline Using AWS

I have mentioned the whole AWS Healthcare ETL project step-by-step as below:

AWS Services Used

- Amazon S3
- AWS Glue Crawler
- AWS Glue Data Catalog
- AWS Glue ETL Job
- IAM Role

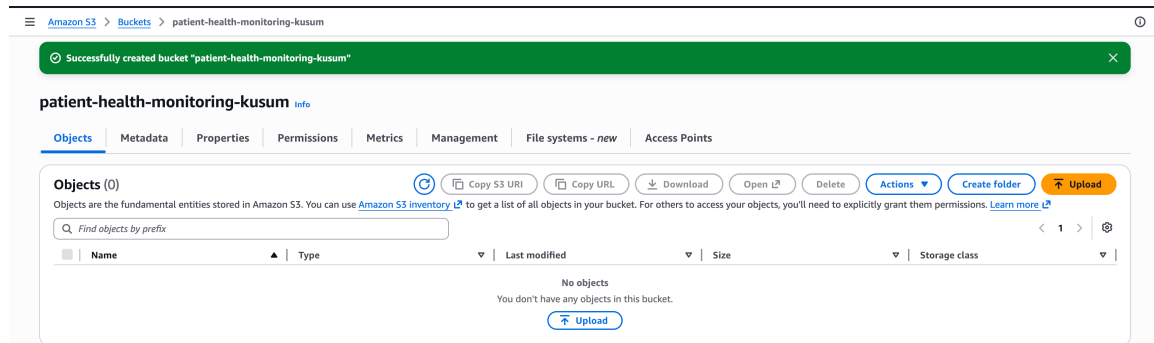
Architecture Flow

CSV Files → Amazon S3 → Glue Crawler → Glue Catalog → Glue ETL Job → Processed Parquet Output

Step 1 – Create S3 Bucket

Open AWS Console → Search S3 → Create Bucket.

Example Bucket Name: patient-health-monitoring-kusum



Step 2 – Create Folder Structure

Create these folders:

```
raw/vitals/  
raw/patients/  
raw/devices/
```

processed/reports/
rejected/
scripts/

Amazon S3 > Buckets > patient-health-monitoring-kusum

Successfully created folder "scripts".

patient-health-monitoring-kusum

Objects Metadata Properties Permissions Metrics Management File systems - new Access Points

Objects (4)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	processed/	Folder	-	-	-
☐	raw/	Folder	-	-	-
☐	rejected/	Folder	-	-	-
☐	scripts/	Folder	-	-	-

Step 3 – Create CSV Files

Create:

1. patient_vitals.csv
2. patient_details.csv
3. device_logs.csv

Amazon S3 > Buckets > patient-health-monitoring-kusum > raw/

raw/

Copy S3 URI

Objects Properties

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	devices/	Folder	-	-	-
☐	patients/	Folder	-	-	-
☐	vitals/	Folder	-	-	-

Step 4 – Upload Files

Upload files into correct S3 folders:

patient_vitals.csv → raw/vitals/
patient_details.csv → raw/patients/
device_logs.csv → raw/devices/

Amazon S3 > Buckets > patient-health-monitoring-kusum > raw/

raw/ Copy S3 URI

Objects Properties

Objects (3) Copy S3 URI Copy URL Download Open L Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory L](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more L](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	devices/	Folder	-	-	-
☐	patients/	Folder	-	-	-
☐	vitals/	Folder	-	-	-

Step 5 – Create IAM Role

Open IAM → Roles → Create Role.

Choose AWS Service → Glue.

Attach:

AmazonS3FullAccess

AWSGlueConsoleFullAccess

Role Name: GlueHealthcareRole

IAM > Roles

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report
- Organization activity
- Service control policies
- Resource control policies

Related consoles

- IAM Identity Center L
- AWS Organizations L

Role GlueHealthcareRole created. View role

Roles (6) Info Delete Create role

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2(S	22 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support(Service-Linker	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor(Service	-
☐	GlueHealthcareRole	AWS Service: glue	-
☐	myLambda	AWS Service: lambda	-
☐	s3-roles	AWS Service: ec2	12 days ago

Roles Anywhere Info Manage

Authenticate your non AWS workloads and securely provide access to AWS services.

Access AWS from your non AWS workloads

Operate your non AWS workloads using the same authentication and authorization strategy that you use within AWS.

X.509 Standard

Use your own existing PKI infrastructure or use [AWS Certificate Manager Private Certificate Authority L](#) to authenticate identities.

Temporary credentials

Use temporary credentials with ease and benefit from the enhanced security they provide.

Step 6 – Create Glue Database

Open AWS Glue → Data Catalog → Databases → Add Database.

Database Name: healthcare_db

AWS Glue > Databases

Databases (1)
A database is a set of associated table definitions, organized into a logical group.

Last updated (UTC)
May 24, 2026 at 04:28:06

[Edit](#) [Delete](#) [Add database](#)

Choose catalog
262892775233

Filter databases

☐	Name	Descrip...	Locatio...	Source ...	Created on (UTC)
☐	healthcare_db	-	-	262892775233	May 24, 2026 at 04:28:06

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)
- Data Catalog**
 - Catalogs
 - Databases**
 - Tables
 - Stream schema registries
 - Schemas
 - Connections
 - Crawlers
 - Classifiers
 - Catalog settings
- Data Integration and ETL**
- Legacy pages**

What's New [L](#)
Documentation [L](#)

Step 7 – Create Glue Crawler

Open AWS Glue → Crawlers → Create crawler.

Add these S3 paths:

s3://patient-health-monitoring-kusum/raw/vitals/

s3://patient-health-monitoring-kusum/raw/patients/

s3://patient-health-monitoring-kusum/raw/devices/

Choose healthcare_db and run crawler.

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)
- Data Catalog**
 - Catalogs
 - Databases
 - Tables
 - Stream schema registries
 - Schemas
 - Connections
 - Crawlers**
 - Classifiers
 - Catalog settings
- Data Integration and ETL**
- Legacy pages**

Step 2: Choose data sources and classifiers

Choose data sources and classifiers

Data source configuration
Is your data already mapped to Glue tables?

☒ Not yet
Select one or more data sources to be crawled.

☐ Yes
Select existing tables from your Glue Data Catalog.

Data sources (3) [Info](#)

The list of data sources to be scanned by the crawler.

Type	Data source	Parameters
☐ S3	s3://patient-heal...	Recrawl all
☐ S3	s3://patient-heal...	Recrawl all
☐ S3	s3://patient-heal...	Recrawl all

[Edit](#) [Remove](#) [Add a data source](#)

Custom classifiers - optional
A classifier checks whether a given file is in a format the crawler can handle. If it is, the classifier creates a schema in the form of a StructType object that matches that data format.

Custom classifiers [Info](#)
Select one or more classifiers to use with this crawler.

[Choose one or more classifiers](#)

[Clear selection](#) [Add new classifier](#) [L](#)

[Cancel](#) [Previous](#) [Next](#)

AWS Glue > **Crawlers** > healthcare_crawler

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers
- Classifiers
- Catalog settings

▼ **Data Integration and ETL**

- Zero-ETL integrations [New](#)
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Interactive Sessions
- Data classification tools
- Sensitive data detection
- Record Matching

healthcare_crawler

One crawler successfully created
The following crawler is now created: "healthcare_crawler"

Last updated (UTC)
May 24, 2026 at 04:47:49 [Refresh](#) [Run crawler](#) [Edit](#) [Delete](#)

Crawler properties

Name healthcare_crawler	IAM role GlueHealthcareRole	Database healthcare_db	State READY
Description -	Security configuration -	Lake Formation configuration -	Table prefix -
Maximum table threshold -			

► **Advanced settings**

[Crawler runs](#) | [Schedule](#) | [Data sources](#) | [Classifiers](#) | [Tags](#)

Crawler runs (0)

The list of crawler runs for this crawler.

[Stop run](#) [View CloudWatch logs](#) [View run details](#)

Start time (UTC)	End time (UTC)	Current/last duration	Status	DPU hours	Table changes
You don't have any crawler runs.					

[Run crawler](#)

Expected Glue Tables

patient_vitals

patient_details

device_logs

AWS Glue > **Tables**

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers
- Classifiers
- Catalog settings

▼ **Data Integration and ETL**

- Zero-ETL integrations [New](#)
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Interactive Sessions
- Data classification tools
- Sensitive data detection

Tables

A table is the metadata definition that represents your data, including its schema. A table can be used as a source or target in a job definition.

Tables (3)

Last updated (UTC)
May 24, 2026 at 04:50:29 [Delete](#) [Add tables using crawler](#) [Add table](#)

View and manage all available tables.

Choose catalog
262892773233

Choose database
healthcare_db

☐	Name	Database	Location	Classific...	Depreca...	View data	Data quality	Column statistics
☐	device_logs_csv	healthcare_db	s3://patient-hea	CSV	-	Table data	View data qual	View statistics
☐	patient_details_	healthcare_db	s3://patient-hea	CSV	-	Table data	View data qual	View statistics
☐	patient_vitals_ci	healthcare_db	s3://patient-hea	CSV	-	Table data	View data qual	View statistics

Step 8 – Create Glue ETL Job

Open AWS Glue → ETL Jobs → Script Editor.

Choose Spark and Python.

Paste PySpark ETL code.

healthcare_etl_job Last modified on 5/24/2026, 12:06:07 AM Actions Save Run

Script | Job details | **Runs** | Data quality | Schedules | Version Control

Job runs (1/3) Info Last updated (UTC) May 24, 2026 at 05:12:40 View details Stop job run Troubleshoot with AI Table View Card View

Filter job runs by property

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacit...	Worker type	Glue version
Succeeded	0	05/24/2026 00:06:09	05/24/2026 00:07:25	1 m 10 s	10 DPU's	G.1X	5.1
Failed	0	05/24/2026 00:00:32	05/24/2026 00:01:34	50 s	10 DPU's	G.1X	5.1
Failed	0	05/23/2026 23:58:44	05/23/2026 23:59:34	41 s	10 DPU's	G.1X	5.1

Run details | Input arguments (11) | Logs | Run insights | Metrics | Troubleshooting analysis | Spark UI

Simple Working ETL Code

```
import sys
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from pyspark.sql.functions import col, when, to_timestamp, to_date, hour

args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args["JOB_NAME"], args)

bucket_name = "patient-health-monitoring-kusum"

vitals_df = spark.read.option("header", True).option("inferSchema", True).csv(
    f"s3://{bucket_name}/raw/vitals/"
)

patients_df = spark.read.option("header", True).option("inferSchema", True).csv(
    f"s3://{bucket_name}/raw/patients/"
)

devices_df = spark.read.option("header", True).option("inferSchema", True).csv(
    f"s3://{bucket_name}/raw/devices/"
)

vitals_df = vitals_df.dropDuplicates(["patient_id", "timestamp"])
```

```

valid_df = vitals_df.filter(
    col("patient_id").isNotNull() &
    col("heart_rate").isNotNull() &
    col("oxygen_level").isNotNull() &
    col("temperature").isNotNull()
)

valid_df = valid_df.withColumn("timestamp", to_timestamp(col("timestamp")))
.validColumn("monitoring_date", to_date(col("timestamp")))
.validColumn("monitoring_hour", hour(col("timestamp")))

valid_df = valid_df.withColumn(
    "health_status",
    when(col("oxygen_level") < 90, "Critical")
    .when(col("temperature") > 100, "Fever")
    .when(col("heart_rate") > 100, "High Risk")
    .otherwise("Normal")
)

final_df = valid_df.join(patients_df, "patient_id", "left") .join(devices_df, "patient_id", "left")

final_df.write.mode("overwrite").parquet(
    f"s3://{bucket_name}/processed/reports/"
)

job.commit()

```

Step 9 – Run Job

Click Save → Run.

Wait until status changes to Succeeded.

Step 10 – Verify Output

Open S3 bucket.
Check processed/reports/ and rejected/ folders.
You should see parquet files.

Amazon S3

Buckets

patient-health-monitoring-kusum

rejected/

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

rejected/

Objects

Properties

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	part-00000-76b34f23-6bc1-41ee-8f90-1db18e135d6d-c000.snappy.parquet	parquet	May 24, 2026, 00:07:14 (UTC-05:00)	1.7 KB	Standard

@Kusum Katwal